UNIT THIRD :- Database Connectivity With MySQL

Python MySQL

MySQL is a widely used open-source relational database for managing structured data. Integrating it with Python enables efficient data storage, retrieval and manipulation within applications.

To work with MySQL in Python, we use **MySQL Connecto**r, a driver that enables seamless integration between the two. It handles the conversion between Python and MySQL data types and is implemented in pure Python, requiring no third-party dependencies.

Why do we need MySQL in Python

1. **Store Large Data:** MySQL helps store and manage large amounts of data efficiently.
2. **Data Persistence:** Data stays saved even after Python program ends.
3. **Easy Data Access:** Python can easily fetch, insert, update or delete data from MySQL.
4. **Supports Multiple Users:** MySQL allows many users to access and work with data at the same time.
5. **Works Well with Python Libraries:** It integrates smoothly with Python tools like Pandas for data analysis.

Installation

To install the **Python-mysql-connector** module, one must have Python and PIP, preinstalled on their system. If Python and pip are already installed type the below command in the terminal.

pip3 install mysql-connector-python

Connecting to MySQL Server

We can connect to both **local** and **cloud-based** MySQL databases using the connect method of mysql-connector library.

Example: Connecting to Local MySQL Database

.

```
# importing required libraries

import mysql.connector

 dataBase = mysql.connector.connect(

  host ="localhost",          # Localhost for local connection

  user ="user",

  passwd ="password"

)
```

```
print(dataBase)

# Disconnecting from the server

dataBase.close()
```

Output:

<mysql.connector.connection_cext.CMySQLConnection object at 0x7f73f0191d00>

Creating Database (CRUD OPERATIONS) [Create (INSERT) ,Read (SELECT),Update,Delete]

After connecting to the MySQL server let's see how to create a MySQL database using Python. For this, we will first create a cursor() object and will then pass the SQL command as a string to the execute() method. The SQL command to create a database is -

CREATE DATABASE DATABASE_NAME

Example: Creating MySQL database with Python

```
# importing required libraries

import mysql.connector

dataBase = mysql.connector.connect(

  host ="localhost",

  user ="user",

  passwd ="password"

)

# preparing a cursor object

cursorObject = dataBase.cursor()

# creating database

cursorObject.execute("CREATE DATABASE gfg")
```

Output:

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| gfg                |
| information_schema |
| mysql              |
| performance_schema |
| sys                |
+--------------------+
5 rows in set (0.10 sec)

mysql>
```

Creating Tables

For creating tables we will follow the similar approach of writing the SQL commands as strings and then passing it to the execute() method of the cursor object. SQL command for creating a table is -

CREATE TABLE
(
column_name_1 column_Data_type,
column_name_2 column_Data_type,
:
:
column_name_n column_Data_type
);

Example: Creating MySQL table using Python

```
# importing required libraries

import mysql.connector



dataBase = mysql.connector.connect(

  host ="localhost",

  user ="user",
```

```
  passwd ="password",
  database = "gfg"
)
# preparing a cursor object
cursorObject = dataBase.cursor()
 # creating table
studentRecord = """CREATE TABLE STUDENT (
                   NAME  VARCHAR(20) NOT NULL,
                   BRANCH VARCHAR(50),
                   ROLL INT NOT NULL,
                   SECTION VARCHAR(5),
                   AGE INT
                   )"""
 # table created
cursorObject.execute(studentRecord)
 # disconnecting from server
dataBase.close()
```

Output:

```
mysql> show tables;
+----------------+
| Tables_in_gfg  |
+----------------+
| STUDENT        |
+----------------+
1 row in set (0.01 sec)

mysql> desc STUDENT;
+---------+--------------+------+-----+---------+-------+
| Field   | Type         | Null | Key | Default | Extra |
+---------+--------------+------+-----+---------+-------+
| NAME    | varchar(20)  | NO   |     | NULL    |       |
| BRANCH  | varchar(50)  | YES  |     | NULL    |       |
| ROLL    | int          | NO   |     | NULL    |       |
| SECTION | varchar(5)   | YES  |     | NULL    |       |
| AGE     | int          | YES  |     | NULL    |       |
+---------+--------------+------+-----+---------+-------+
5 rows in set (0.00 sec)
```

Inserting Data

it is required to make sure there is no mismatch of data i.e. the data type being sent should match the data type of the particular column.

The general syntax for insert statements:

```
INSERT INTO <TABLE_NAME> (column1,column2,column3...) VALUES (data1,data2,data3...);
```

We will be inserting multiple rows at one type, however, you can even insert one row at a time. After writing the insert statement, we will be creating a list or collections of row data to be passed. This is to be created right before the executed query.

Since multiple rows will be sent together, we need to use the executemany() function instead of execute().

```
# Python implementation to insert data into a table in MySQL

import mysql.connector

# connecting to the mysql server

db = mysql.connector.connect(

    host="localhost",

    user="root",

    passwd="password",

    database="employee_db"

)

# cursor object c

c = db.cursor()

# insert statement for tblemployee

# this statement will enable us to insert multiple rows at once.

employeetbl_insert = """INSERT INTO tblemployee (

   empname,

   department,
```

```
  salary)
  VALUES  (%s, %s, %s)"""
# we save all the row data to be inserted in a data variable
data = [("Vani", "HR", "100000"),
       ("Krish", "Accounts", "60000"),
       ("Aishwarya", "Sales", "25000"),
       ("Govind", "Marketing", "40000")]
# execute the insert commands for all rows and commit to the database
c.executemany(employeetbl_insert, data)
db.commit()
# finally closing the database connection
db.close()
```

Reading / Selecting Data

Reading/Selecting or fetching data from the database follows the command:

```
SELECT * FROM <TABLE_NAME>
```

This command fetches all columns/attributes from the table.

However at times, one may need to fetch only certain columns. For that we run the following command:

```
SELECT COLUMN1,COLUMN2... FROM <TABLE_NAME>
```

Let us look at the implementation for selecting all columns.

```
# Python implementation to fetch data from a table in MySQL
import mysql.connector


# connecting to the mysql server
```

```
db = mysql.connector.connect(
    host="localhost",
    user="root",
    passwd="password",
    database="employee_db"
)

# cursor object c
c = db.cursor()

# select statement for tblemployee which returns all columns
employeetbl_select = """SELECT * FROM tblemployee"""

# execute the select query to fetch all rows
c.execute(employeetbl_select)

# fetch all the data returned by the database
employee_data = c.fetchall()

# print all the data returned by the database
for e in employee_data:
    print(e)

# finally closing the database connection
```

```
db.close()
```

Output:

```
(1, 'Vani', 'HR', 100000)
(2, 'Krish', 'Accounts', 60000)
(3, 'Aishwarya', 'Sales', 25000)
(4, 'Govind', 'Marketing', 40000)
```

Employee records fetched from tblemployee

Updating Data

Updating data is changing the existing values in the database. Let's say in our employee records, the salary of an employee named "Vani" has been increased by 15% hence the salary in the records should become 115000. Hence we will run the update query whose general syntax is:

```
UPDATE <TABLE_NAME> SET <COLUMN_NAME> = <VALUE> WHERE <PRIMARY KEY
NAME> =<PRIMARY KEY VALUE>
```

```
# Python implementation to update data of a table in MySQL

import mysql.connector

# connecting to the mysql server

db = mysql.connector.connect(

    host="localhost",

    user="root",

    passwd="password",

    database="employee_db"

)

# cursor object c

c = db.cursor()

# update statement for tblemployee

# which modifies the salary of Vani

employeetbl_update = "UPDATE tblemployee\
```

```
SET salary = 115000 WHERE empid = 1"

# execute the update query to modify

# the salary of employee with

# employee id = 1 and commit to the database

c.execute(employeetbl_update)

db.commit()

# finally closing the database connection

db.close()
```

empid	empname	department	salary
1	Vani	HR	115000
2	Krish	Accounts	60000
3	Aishwarya	Sales	25000
4	Govind	Marketing	40000
NULL	NULL	NULL	NULL

Deleting Data

Deleting data from tables has to be done with utmost care as it can lead to the loss of important data at times. Often a soft delete is performed where there is an extra column named "active" whose values are 1 or 0 only. 1 means present in the table and 0 means deleted from being displayed i.e. it is still recoverable but not shown or acted upon.

However, in this tutorial, we will be performing a regular or hard delete whose general syntax is:

```
DELETE FROM <TABLE_NAME> WHERE <PRIMARY KEY NAME> = <PRIMARY KEY
VALUE>
```

The where clause doesn't necessarily have to be the primary key.

We will be deleting Aishwarya(empid=3) from our records.

```
import mysql.connector

# connecting to the mysql server
```

```
db = mysql.connector.connect(
    host="localhost",
    user="root",
    passwd="password",
    database="employee_db"
)
# cursor object c
c = db.cursor()
# delete statement for tblemployee
# which deletes employee Aishwarya having empid 3
employeetbl_delete = "DELETE FROM tblemployee WHERE empid=3"
# execute the delete statement and commit to the database
c.execute(employeetbl_delete)
db.commit()
# finally closing the database connection
db.close()
```

	empid	empname	department	salary
▶	1	Vani	HR	115000
	2	Krish	Accounts	60000
	4	Govind	Marketing	40000
*	NULL	NULL	NULL	NULL

Python Tkinter

Tkinter is Python's built-in library for creating graphical user interfaces (GUIs). It acts as a lightweight wrapper around Tcl/Tk GUI toolkit, offering Python developers a simple and intuitive way to build desktop applications. It supports layout management, event handling and customization, making it ideal for rapid GUI development in Python.

Why do we need Tkinter?

1. Provides a built-in and easy-to-use way to create GUI (Graphical User Interface) applications in Python.
2. Offers various widgets like buttons, labels, text boxes and menus to build interactive apps.
3. Eliminates the need for external GUI frameworks tkinter comes bundled with Python.
4. Useful for building desktop applications like calculators, form apps and dashboards.
5. Supports event-driven programming, making it suitable for responsive user interfaces.

Create First Tkinter GUI Application

To create a Tkinter Python app, follow these basic steps:

1. **Import the tkinter module**: Import the tkinter module, which is necessary for creating the GUI components.
2. **Create the main window (container)**: Initialize the main application window using the Tk() class.
3. **Set Window Properties**: We can set properties like the title and size of the window.
4. **Add widgets to the main window**: We can add any number of widgets like buttons, labels, entry fields, etc., to the main window to design the interface.
5. **Pack Widgets:** Use geometry managers like pack(), grid() or place() to arrange the widgets within the window.
6. **Apply event triggers to the widgets**: We can attach event triggers to the widgets to define how they respond to user interactions.

There are two main methods used which user needs to remember while creating the Python application with GUI. These methods are:

1. Tk()

To create a main window in Tkinter, we use the Tk() class.

Syntax:

root = Tk(screenName=None, baseName=None, className='Tk', useTk=1)

Parameter:

- **screenName (optional):** Specifies the display (mainly used on Unix systems with multiple screens).
- **baseName (optional):** Sets the base name for the application (default is the script name).
- **className (optional):** Defines the name of the window class (used for styling and window manager settings).

- **useTk (optional):** A boolean that tells whether to initialize the Tk subsystem (usually left as default 1).

2. mainloop()

The mainloop() method is used to run application once it is ready. It is an infinite loop that keeps application running, waits for events to occur (such as button clicks) and processes these events as long as window is not closed.

Example: This code initializes a basic GUI window using Tkinter. The tkinter.Tk() function creates main application window, mainloop() method starts event loop, which keeps the window running and responsive to user interactions.

```
import tkinter

m = tkinter.Tk()

'''

widgets are added here

'''

m.mainloop()
```

Tkinter Widget

There are a number of tkinter widgets which we can put in our tkinter application. Some of the major widgets are explained below:

1. Label

It refers to the display box where we display text or image. It can have various options like font, background, foreground, etc.

Syntax:

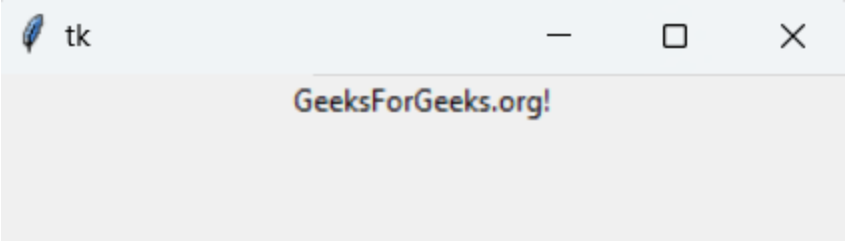
w=Label(master, option=value)

Parameter: master used to represent parent window.

Example: This code creates a simple GUI window using Tkinter that displays a text label. It imports all Tkinter classes, initializes main window (root), adds a Label widget with text "GeeksForGeeks.org!", packs label into window.

```
from tkinter import *

root = Tk()
```

```
w = Label(root, text='GeeksForGeeks.org!')

w.pack()

root.mainloop()
```



2. Button

A clickable button that can trigger an action.

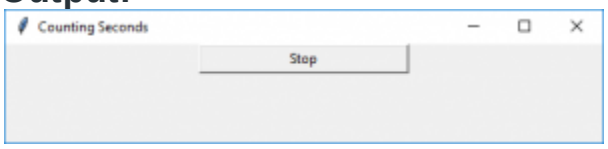
Syntax:

w=Button(master, option=value)

Example: This code creates a window titled "Counting Seconds" with a single "Stop" button. When button is clicked, window closes. It demonstrates how to set a window title, create a button widget and assign a command to terminate the application.

```
import tkinter as tk

r = tk.Tk()

r.title('Counting Seconds')

button = tk.Button(r, text='Stop', width=25, command=r.destroy)

button.pack()

r.mainloop()
```

Output:



3. Entry

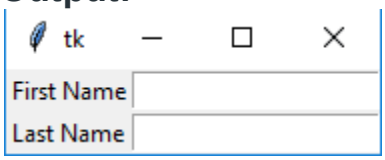
It is used to input the single line text entry from the user. For multi-line text input, Text widget is used.

Syntax:

w=Entry(master, option=value)

Example: This code creates a form with two labeled input fields: First Name and Last Name. It uses Label widgets to display field names and Entry widgets to accept user input. The grid() geometry manager arranges the labels and entry fields in a tabular layout

Output:



4. CheckButton

A checkbox can be toggled on or off. It can be linked to a variable to store its state.

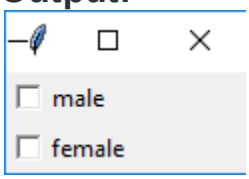
Syntax:

w = CheckButton(master, option=value)

Example: This code creates a window with two checkboxes labeled "male" and "female". It uses Checkbutton widgets linked to IntVar() variables to store checked state (1 for checked, 0 for unchecked). The grid() method arranges checkboxes vertically with left alignment using sticky=W.

```
from tkinter import *

master = Tk()

var1 = IntVar()

Checkbutton(master, text='male', variable=var1).grid(row=0, sticky=W)

var2 = IntVar()

Checkbutton(master,text='female',variable=var2).grid(row=1, sticky=W)

mainloop()
```

Output:



5. RadioButton

It allows user to select one option from a set of choices. They are grouped by sharing the same variable.

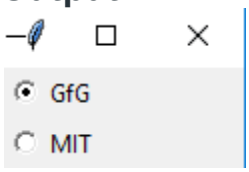
Syntax:

w = RadioButton(master, option=value)

Example: This code creates a window with two radio buttons labeled "GfG" and "MIT". Radio buttons are linked to same IntVar() variable v ensuring only one can be selected at a time. The selected option sets v to its respective value (1 or 2) and buttons are vertically aligned to left using pack(anchor=W).

```
from tkinter import *

root = Tk()

v = IntVar()

Radiobutton(root, text='GfG', variable=v, value=1).pack(anchor=W)

Radiobutton(root, text='MIT', variable=v, value=2).pack(anchor=W)

mainloop()
```

Output



6. Listbox

It displays a list of items from which a user can select one or more.

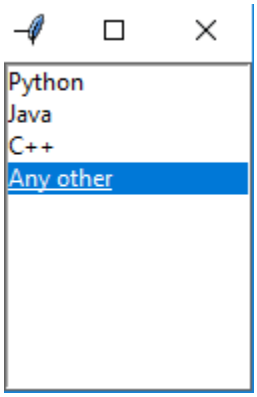
Syntax:

w = Listbox(master, option=value)

Example: This code creates a window with a Listbox widget that displays a list of programming languages. The insert() method adds each item to Listbox at specified index and pack() displays Listbox in the window. The GUI runs using mainloop(

```
from tkinter import *

top = Tk()

Lb = Listbox(top)

Lb.insert(1, 'Python')

Lb.insert(2, 'Java')

Lb.insert(3, 'C++')

Lb.insert(4, 'Any other')
```

```
Lb.pack()

top.mainloop()
```



UNIT IV -----Web Development using Python

Django Installation

Installing and setting up Django is a straightforward process. Below are the step-by-step instructions to install Django and set up a new Django project on your system.

Step 1: Install Pip

Open the command prompt and enter the following command-
python -m pip install -U pip

```
C:\Windows\System32>python -m pip install -U pip
Requirement already satisfied: pip in c:\program file
Collecting pip
  Using cached pip-24.0-py3-none-any.whl.metadata (3
Using cached pip-24.0-py3-none-any.whl (2.1 MB)
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 23.2.1
    Uninstalling pip-23.2.1:
      Successfully uninstalled pip-23.2.1
Successfully installed pip-23.3.2
```